# Package 'nomeR'

May 15, 2026

**Type** Package

**Title** Probabilistic framework for analysis of single-molecule footprinting data

**Date** 2025-05-16

**Version** 0.9.3

**Description** \code{nomeR} uses a probabilistic model for analysis of data from single-molecule footprinting (SMF) assays like NOMe-Seq that allow to acquire single-molecule positional information about footprints of DNA binding proteins.

**License** MIT + file LICENSE

**Imports** checkmate, cli, data.table, dplyr, GenomeInfoDb, GenomicRanges, ggplot2, graphics, gtools, IRanges, magrittr, methods, parallel, patchwork, Rcpp (>= 0.12.17), RcppParallel (>= 5.0.1), rlang, rstan (>= 2.18.1), rstantools (>= 2.4.0), S4Vectors, Seqinfo, SparseArray, stats, stringr, SummarizedExperiment, truncdist

**Suggests** testthat (>= 3.0.0)

**LinkingTo** BH (>= 1.66.0), Rcpp (>= 0.12.0), RcppEigen (>= 0.3.3.3.0), RcppParallel (>= 5.0.1), RcppProgress, rstan (>= 2.18.1), StanHeaders (>= 2.18.0)

**Encoding** UTF-8

**Roxygen** list(markdown = TRUE)

**Biarch** true

**Depends** R (>= 3.5)

**SystemRequirements** GNU make

**Config/testthat/edition** 3

**URL** https://github.com/fmi-basel/compbio-nomeR

**BugReports** https://github.com/fmi-basel/compbio-nomeR/issues

**biocViews** Epigenetics, LongRead, Sequencing, Coverage

**Config/roxygen2/version** 8.0.0

**RoxygenNote** 7.3.3

**NeedsCompilation** yes

**Author** Evgeniy A. Ozonov [aut, cre]

**Maintainer** Evgeniy A. Ozonov <evgeniy.ozonov@fmi.ch>

# Contents



---

`nomeR-package` *The 'nomeR' package.*

---

## Description

Regulation of gene expression by DNA binding proteins, such as sequence specific transcription factor (TF), is a key step which drives a plethora of biological processes. During the past decade, remarkable achievements have been made in understanding how TFs regulate their targets using ChIP-seq and ATAC-seq assays. However, these assays provide only averaged information of larger cell populations. Hence, such assays are unable to detect states of binding at single-DNA-molecule resolution. In contrast, emerging single-molecule footprinting (SMF) technologies, such as the Nucleosome Occupancy and Methylome Sequencing (NOMe-seq) assay, provide information about states of binding sites at single-DNA-molecule resolution and allow better molecular characterization of inherently stochastic processes of protein-DNA interactions.

Despite the many advantages that single molecule footprinting technologies bring for studying mechanisms of gene regulation, rigorous statistical models for the analysis of datasets generated using these technologies are still missing. Here, we present, to our knowledge, the first statistical model for inference and predictions of footprints for unbiased quantitative analysis and interpretation of SMF datasets.

## Author(s)

**Maintainer**: Evgeniy A. Ozonov `<evgeniy.ozonov@fmi.ch>`

## References

Stan Development Team (NA). RStan: the R interface to Stan. R package version 2.32.6. https://mc-stan.org

## See Also

Useful links:

- https://github.com/fmi-basel/compbio-nomeR
- Report bugs at https://github.com/fmi-basel/compbio-nomeR/issues

---

`.quantile_normalise` *Standard quantile normalisation*

---

## Description

Standard quantile normalisation

## Usage

```
.quantile_normalise(p_z1, y_raw)
```

## Arguments

| | |
|---|---|
| `p_z1` | Numeric vector: posterior p(Z=1) (to be normalised) |
| `y_raw` | Numeric vector: reference distribution (raw scores) |

## Value

Numeric vector with distribution matched to y_raw

---

`calculate_BG_TF_scores_SE` *Compute isometric log-ratio scores from predicted posterior coverages*

---

## Description

Compute isometric log-ratio scores from predicted posterior coverages

## Usage

```
calculate_BG_TF_scores_SE(
  se,
  bgAssayNames = c("background_coverProb_nomeR"),
  tfAssayNames = c("TF_coverProb_nomeR"),
  nuclAssayNames = c("Nucl_coverProb_nomeR"),
  mod_probAssayName = "mod_prob",
  psc = 0.1
)
```

### Arguments

| | |
|---|---|
| se | A SummarizedExperiment object containing read-level data, such as that returned by SingleMoleculeGenomicsIO::readModBam, including modification probabilities. |
| bgAssayNames | Character vector of assay names containing predicted coverage probabilities for background (accessible state). |
| tfAssayNames | Character vector of assay names containing predicted coverage probabilities for transcription factors. |
| nuclAssayNames | Character vector of assay names containing predicted coverage probabilities for nucleosomes. |
| mod_probAssayName | Character scalar. Name of the assay containing raw modification probabilities. |
| psc | Numeric. Pseudo-count added before taking logarithms to avoid log(0). |

### Value

A SummarizedExperiment object with BG and TF scores stored as additional assays.

---

## calculate_tile_BG_TF_enrichments

*Enrichment of Accessible Sites and TF Footprints Across Genomic Tiles*

---

### Description

Calculates enrichments and statistical significance for accessible positions (background) and transcription factor (TF) footprints across genomic tiles.

### Usage

```
calculate_tile_BG_TF_enrichments(
  cover_dt,
  tile_width = 500,
  tile_step = 250,
  bg_colname = "background",
  tf_colname = "TF",
  nucl_colname = "Nucl",
  psc = 0.1,
  threads = NULL
)
```

### Arguments

| | |
|---|---|
| cover_dt | A data.table containing posterior coverages returned by predict_footprints_SE with returnAs="data.table". |
| tile_width | Width of the sliding windows (tiles). |
| tile_step | Step size for sliding windows. |

| | |
|---|---|
| bg_colname | Name of the column containing posterior coverage for background (accessible positions). |
| tf_colname | Name of the column containing posterior coverage for TF footprints. |
| nucl_colname | Name of the column containing posterior coverage for nucleosome footprints. |
| psc | Pseudo-count added when calculating background and TF scores. |
| threads | Number of threads used by data.table. NULL (default) re-reads environment variables. 0 uses all available logical CPUs. Otherwise a positive integer. |

## Value

A data.table with enrichment values and statistical significance calculated using a Z-test on BG and TF scores. The null hypothesis is that the mean BG/TF score for each tile equals the mean across all tiles. The table contains the following columns:

**seqnames, start, end, tileID** Genomic coordinates of sliding windows (tiles).

**n_data_points** Number of informative positions with modification data aggregated across all molecules.

**bg_score_mean, tf_score_mean** Mean background and TF scores across all molecules and informative positions overlapping each tile.

**bg_pos_cnt, tf_pos_cnt** Number of positions classified as "positive" for accessibility (background) or TF footprints, after applying bg_score_thresh and tf_score_thresh, aggregated across all molecules overlapping the tile.

**bg_log2enr, tf_log2enr** Log2 ratios of observed over expected counts for accessible and TF-covered positions.

**bg_binom_pval, tf_binom_pval, bg_FDR, tf_FDR** P-values (and FDR-adjusted p-values) from one-tailed binomial tests (binom.test) under the null hypothesis that the fraction of accessible or TF-covered positions in each tile equals the genome-wide expected fraction. The alternative hypothesis is that the observed fraction is greater than expected. Adjusted p-values are computed using p.adjust with method="fdr".

---

# correct_modprob_SE *Correct modification probabilities for sequence-context bias*

---

## Description

Applies a per-position Bayesian correction to raw modification probabilities using Beta distributions fitted to negative-control (unmodified) and positive-control (fully modified) samples, stratified by sequence context.

## Usage

```
correct_modprob_SE(
  se,
  assayName = "mod_prob",
  corrected_assayName = "mod_prob_corrected",
  neg_control_shapes,
  pos_control_shapes,
  mod_prior = 0.5,
  eps = .Machine$double.eps,
  qnorm_to_raw = FALSE
)
```

### Arguments

| | |
|---|---|
| se | A SummarizedExperiment object containing read-level data, such as that returned by SingleMoleculeGenomicsIO::readModBam, including modification probabilities. |
| assayName | Character scalar specifying the name of the assay in se that contains read-level modification probabilities. |
| corrected_assayName | Character scalar. Name of the additional assay added to the returned se that will contain the corrected read-level modification probabilities. |
| neg_control_shapes | A data.table containing Beta distribution shape parameters for the negative control experiment (no MTase treatment), as returned by get_SeqContext_control_beta_sh |
| pos_control_shapes | A data.table containing Beta distribution shape parameters for the positive control experiment (MTase treatment of naked DNA), as returned by get_SeqContext_control |
| mod_prior | prior probability of a modified base. |
| qnorm_to_raw | if TRUE perform quantile normalization of the corrected probabilities to match the distribution of the raw probabilities. |

### Value

The input se with an additional assay (named by corrected_assayName) containing the corrected modification probabilities.

---

## extract_assay_datatable

*Extract assay data from a SummarizedExperiment as a* data.table

---

### Description

Extract assay data from a SummarizedExperiment as a data.table

### Usage

```
extract_assay_datatable(se, sampleName, assayName, datColName = NULL)
```

### Arguments

| | |
|---|---|
| se | A SummarizedExperiment object containing read-level data, such as that returned by SingleMoleculeGenomicsIO::readModBam, including modification probabilities. |
| sampleName | Character scalar. Name of the sample to extract data for. |
| assayName | Character scalar. Name of the assay to extract. |
| datColName | Character scalar. Name to use for the data column in the output data.table. Defaults to "assay_data" if NULL. |

## Value

A `data.table` with columns:

`seqnames` Chromosome name.

`refPos` Genomic coordinate.

`fragID` Fragment (read) name.

`assay_data` Data values (renamed to `datColName` if supplied).

---

`ftp_spectral_analysis_SE` *Footprint spectral analysis for a SummarizedExperiment containing single-molecule footprinting (SMF) data*

---

## Description

Performs footprint spectral analysis on a `SummarizedExperiment` object containing single-molecule footprinting (SMF) data. For each sample, the function extracts pair-state statistics, computes co-occurrence tables, and infers footprint spectra and emission probabilities using the specified Bayesian inference method.

## Usage

```
ftp_spectral_analysis_SE(
  se,
  assayName = "mod_prob",
  threshMod = 0.5,
  threshUnmod = threshMod,
  min_frag_data_len = 50L,
  min_frag_data_dens = 0.05,
  max_spacing = 200,
  ftp_lengths = 20:200,
  ftp_prior_cover = NULL,
  bg_prior_cover = 0.5,
  total_cnt_prior_dirich = NULL,
  ftp_bg_model = c("informative_prior", "bg_fixed", "ftp_bg_fixed"),
  bg_model_params = list(bg_protect_prob_fixed = 0.05, bg_protect_min = 0.01,
    bg_protect_max = 0.2, bg_protect_mean = 0.05, bg_protect_totcount = 100),
  ftp_model_params = list(ftp_protect_prob_fixed = 0.95, ftp_protect_min = 0.8,
    ftp_protect_max = 0.99, ftp_protect_mean = 0.95, ftp_protect_totcount = 100),
  max_nruns = 3,
  max_pareto_k = 10,
  ncpu = 1L,
  verbose = FALSE,
  iter = 15000,
  tol_rel_obj = 1e-08,
  output_samples = 2000,
  grad_samples = 1,
  algorithm = "meanfield",
  ...
)
```

## Arguments

| | |
|---|---|
| se | A SummarizedExperiment object containing read-level data, such as that returned by SingleMoleculeGenomicsIO::readModBam, including modification probabilities. |
| assayName | Character scalar specifying the name of the assay in se that contains read-level modification probabilities. |
| threshUnmod, threshMod | Numeric thresholds used to binarize modification probabilities into accessible (0; probability >= threshMod), protected (1; probability < threshUnmod), or unknown (NA) states. |
| min_frag_data_len | Ignore fragments whose genomic span (from the leftmost to rightmost non-NA data point) is shorter than min_frag_data_len. |
| min_frag_data_dens | Ignore fragments with a density of informative (non-NA) positions below min_frag_data_dens. |
| max_spacing | Integer specifying the maximum spacing (in bases) between positions when counting co-occurrences of state pairs (00, 01, 10, 11) across the SMF dataset. |
| ftp_lengths | Numeric vector of footprint lengths for which abundances should be inferred. Length 1 is reserved for background and must not be included. |
| ftp_prior_cover | Numeric vector of expected (prior) coverages for the footprint lengths in ftp_lengths. These values define the prior mean of the Dirichlet distribution over footprint/background abundances. All elements must be in [0, 1]. Together with bg_prior_cover, they will be scaled to sum to 1 if necessary (with a warning). |
| bg_prior_cover | Numeric value in [0, 1] specifying the expected (prior) fraction of background (unprotected) positions. Used together with ftp_prior_cover to parameterize the prior Dirichlet distribution. |
| total_cnt_prior_dirich | Numeric value giving the total count parameter of the prior Dirichlet distribution. Higher values impose stronger prior concentration around the mean (i.e., stronger prior beliefs), while lower values yield a more diffuse prior. |
| ftp_bg_model | Character string specifying the modeling strategy:<br>"informative_prior" Infer bg_protect_prob, ftp_protect_prob, and footprint abundances.<br>"bg_fixed" Fix bg_protect_prob to the value in bg_model_params[["bg_protect_ and infer ftp_protect_prob and footprint abundances.<br>"ftp_bg_fixed" Fix both bg_protect_prob and ftp_protect_prob (values in bg_model_params and ftp_model_params). Only footprint abundances are inferred. |
| bg_model_params | A list of background model parameters:<br>**bg_protect_prob_fixed** Fixed value for bg_protect_prob used in "bg_fixed" and "ftp_bg_fixed" models.<br>**bg_protect_min, bg_protect_max** Lower and upper bounds for bg_protect_prob when it is inferred. Ignored in fixed models.<br>**bg_protect_mean** Mean of the prior Beta distribution for bg_protect_prob. Ignored when fixed. |

**bg_protect_totcount** Total count parameter of the prior Beta distribution (controls concentration). Ignored when fixed.

`ftp_model_params`
: A list of footprint model parameters:

**ftp_protect_prob_fixed** Fixed value for `ftp_protect_prob` (used only in `"ftp_bg_fixed"`).

**ftp_protect_min, ftp_protect_max** Bounds for `ftp_protect_prob` when it is inferred. Ignored in `"ftp_bg_fixed"`.

**ftp_protect_mean** Mean of the prior Beta distribution for `ftp_protect_prob`. Ignored when fixed.

**ftp_protect_totcount** Total count parameter of the prior Beta distribution. Ignored when fixed.

`max_nruns`
: Maximum number of attempts to run `vb`. Some initializations may fail to converge, in which case multiple attempts are made.

`max_pareto_k`
: Maximum allowable `pareto_k` value returned by `vb`. If exceeded, inference is retried (up to `max_nruns` times).

`ncpu`
: Number of threads to use.

`verbose`
: Logical. If `TRUE`, enables verbose output for debugging.

`iter`, `tol_rel_obj`, `output_samples`, `grad_samples`, `algorithm`, `...`
: Additional arguments passed to `vb`, controlling ADVI inference. See `vb` for details.

## Value

A `DataFrame` (from `colData(se)`) augmented with additional columns containing:

- pair-state summary statistics,
- inferred footprint spectra,
- estimated emission probabilities.

Each row corresponds to a sample in the input `SummarizedExperiment`.

---

`generate_insilico_SMF_data`
*Generate an in-silico single-molecule footprinting dataset*

---

## Description

Generate an in-silico single-molecule footprinting dataset

## Usage

```
generate_insilico_SMF_data(
  region_len,
  n_reads,
  footprint_models,
  ftp_emission_posprob = NULL,
  bgprotectprob,
  infposdens = 1
)
```

## Arguments

`region_len` Integer. Length of the simulated region (number of positions).

`n_reads` Integer. Number of molecules to generate.

`footprint_models`
A `list` describing the footprints to generate. Each element is itself a `list` that must contain:

`PROTECT_PROB` Numeric vector of emission probabilities for protected positions, with length equal to the footprint length.

`COVER_PROB` Numeric scalar giving the prior coverage probability for the footprint.

`NAME` Character scalar: name of the footprint.

`ftp_emission_posprob`
A numeric `matrix` of positional start probabilities for each footprint. The number of rows must equal the number of footprints and the number of columns must equal `region_len`.

`bgprotectprob`
Numeric. Emission probability for background (accessible) positions.

`infposdens` If a scalar in `(0, 1]`, treated as the proportion of positions that are informative (observable). If a vector of integers, treated as the explicit indices of informative positions.

## Value

A `list` containing:

`DATA` A `matrix` with the simulated SMF data.

`TRUE_CONF` A `data.frame` with the true footprint positions for each molecule.

`ftp_pos_prob` Normalized positional start probabilities used for footprint placement.

## Examples

```
## length of a chromosome
chr_len <- 1000

ftp_cover_c2 <- c(0.05, 0.5)
ftp_len_c2 <- c(50, 150)
case2_pos_mat <- rbind(matrix(c(rep(0, 515), 1, rep(0, 484)),
                              nrow = 1,
                              ncol = chr_len),
                       matrix(1 / chr_len,
                              nrow = 1,
                              ncol = chr_len))

## sequencing depths
max_nreads <- 30
## footprint model
case2_ftp_models <- list(
"ftp50" = list("PROTECT_PROB" = rep(1,ftp_len_c2[1]),
               "COVER_PRIOR" = ftp_cover_c2[1],
               "NAME" = "ftp50"),
"ftp150" = list("PROTECT_PROB" = rep(1,ftp_len_c2[2]),
                "COVER_PRIOR" = ftp_cover_c2[2],
```

```
                  "NAME" = "ftp150"))

  smf_data <-
    generate_insilico_SMF_data(region_len = chr_len,
                               n_reads = max_nreads,
                               footprint_models = case2_ftp_models,
                               ftp_emission_posprob = case2_pos_mat,
                               bgprotectprob = 0,
                               infposdens = 1
                               )
```

---

`get_ctable_from_matrix` *Create co-occurrence count tables (00, 01, 10, 11) from a binary SMF data matrix*

---

## Description

Computes co-occurrence frequencies of binary accessibility states (`00`, `01`, `10`, `11`) at varying spacings within a matrix representing a single-molecule footprinting (SMF) region of interest (ROI). Rows correspond to individual molecules (or fragments) and columns correspond to genomic positions. In the matrix, `1` represents a protected position and `0` represents an accessible position.

## Usage

```
get_ctable_from_matrix(data, max_spacing = 200L, ncpu = 1L, verbose = FALSE)
```

## Arguments

| | |
|---|---|
| `data` | A binary matrix of SMF data for a single ROI. |
| `max_spacing` | Integer specifying the maximum spacing (in bases) between positions for which co-occurrence frequencies are computed. |
| `ncpu` | Number of CPU cores to use for parallel computation. |
| `verbose` | Logical. If `TRUE`, prints progress and diagnostic messages for debugging. |

## Value

A `data.frame` containing co-occurrence frequencies for `00`, `01`, `10`, and `11` at each spacing from 1 up to `max_spacing`.

get_ctable_from_SE *Create count tables of* 00, 01, 10, *and* 11 *co-occurrences in a SummarizedExperiment object containing SMF modification probabilities*

## Description

Computes co-occurrence frequencies of binary accessibility states (00, 01, 10, 11) at varying spacings within a SummarizedExperiment object containing single-molecule footprinting (SMF) modification probabilities. Frequencies are calculated for distances up to max_spacing, either per sample or aggregated across all samples.

## Usage

```
get_ctable_from_SE(
  se,
  assayName = "mod_prob",
  threshMod = 0.5,
  threshUnmod = threshMod,
  min_frag_data_len = 50L,
  min_frag_data_dens = 0.05,
  max_spacing = 200L,
  aggrSamples = FALSE,
  ncpu = 1L,
  verbose = FALSE
)
```

## Arguments

| | |
|---|---|
| se | A SummarizedExperiment object containing read-level data, such as that returned by SingleMoleculeGenomicsIO::readModBam, including modification probabilities. |
| assayName | Character scalar specifying the name of the assay in se that contains read-level modification probabilities. |
| threshUnmod, threshMod | Numeric thresholds used to binarize modification probabilities into accessible (0; probability >= threshMod), protected (1; probability < threshUnmod), or unknown (NA) states. |
| min_frag_data_len | Ignore fragments whose genomic span (from the leftmost to rightmost non-NA data point) is shorter than min_frag_data_len. |
| min_frag_data_dens | Ignore fragments with a density of informative (non-NA) positions below min_frag_data_dens. |
| max_spacing | Integer specifying the maximum spacing (in bases) between positions when counting co-occurrences of state pairs (00, 01, 10, 11) across the SMF dataset. |
| aggrSamples | Logical. If TRUE, co-occurrence frequencies are aggregated across all samples. If FALSE, a separate frequency matrix is returned for each sample. |
| ncpu | Number of CPU cores to use for parallel processing. |
| verbose | Logical. If TRUE, prints additional progress messages for debugging. |

### Value

If `aggrSamples = TRUE`, a `matrix` containing aggregated co-occurrence frequencies for `00`, `01`, `10`, and `11` at spacings from 1 to `max_spacing`.

If `aggrSamples = FALSE`, a `list` of matrices, one per sample, where each matrix contains the corresponding co-occurrence frequencies.

---

`get_ftp_inference_summary`
*Extract and/or plot estimates from footprint inference and optionally suggest footprints*

---

### Description

Utility function to extract footprint abundance estimates from a `stanfit` or optimization result (from `infer_footprints_vb`, `infer_footprints_sampling`, or `infer_footprints_optim`) and, optionally, detect potential footprints from the abundance spectrum using `suggest_footprints`.

### Usage

```
get_ftp_inference_summary(
  infer_stanfit,
  plot = FALSE,
  show_plot = plot,
  suggest_ftps = FALSE,
  plot_posterior_range = c("2.5%", "97.5%"),
  max_peak_width = 30,
  spline_spar = 0.6,
  max_abund_log2drop = 1.5,
  ftp_abundance_name = c("ftp_abundances"),
  ...
)
```

### Arguments

`infer_stanfit`
: A `stanfit` object returned by `infer_footprints_vb` or `infer_footprints_sampling` or a `list` returned by `infer_footprints_optim`.

`plot`
: Logical. If `TRUE`, generate a plot of the footprint abundance spectrum.

`show_plot`
: Logical. If `TRUE`, display the plot of the footprint spectrum.

`suggest_ftps`
: Logical. If `TRUE`, return suggested footprints based on spectrum peaks.

`plot_posterior_range`
: Character vector of length 2 specifying which posterior credible intervals to plot. Options provided by `summary,stanfit-method` include "2.5\ Standard deviation (SD) and standard error (SE) are not currently supported.

`max_peak_width`
: Numeric. Maximum allowed width of detected peaks in the footprint abundance spectrum (used by `suggest_footprints`).

`spline_spar` Numeric in (0, 1]. Smoothing parameter for `suggest_footprints` controlling the smoothness of the spline (`smooth.spline`). Recommended to test values like 0.1, 0.3, 0.5, 0.75 to ensure reasonable footprint detection.

`max_abund_log2drop` Numeric. Maximum decrease in log2 abundance relative to the local maximum when extending peaks in `suggest_footprints`.

`ftp_abundance_name` Character scalar specifying which footprint abundance value to extract or display. Currently only `"ftp_abundances"` is supported.

`...` Additional parameters passed to `suggest_footprints`, including those for `smooth.spline`.

## Value

A `list` containing:

`ESTIMATES` A `list` containing:

- `ftp_abundance_estimates`: `data.frame` of footprint abundance estimates,
- `ftp_protect_prob_estimate`: estimated footprint protection probability (or `NA` if unavailable),
- `bg_protect_prob_estimate`: estimated background protection probability (or `NA` if unavailable).

`FTP_SUGGEST` `matrix` with coordinates of suggested footprints returned by `suggest_footprints`.

`PLOT` `ggplot` object of the footprint abundance spectrum.

## Examples

```
## Simple data with two footprints of lengths 5 and 10 bp.
## The table below is a count table of observed occurrences of
## 00, 01, 10, and 11 at spacings from 1 to 15.

ftp_5_10_data <- data.frame("S" = 1:15,
                            "N00" = c(1626964, 1508381,
                                      1420066, 1336897,
                                      1258679, 1185045,
                                      1136784, 1090029,
                                      1045066, 1001670,
                                      959927, 983020,
                                      1000410, 1012326,
                                      1019633),
                            "N01" = c(0, 113856,
                                      197657, 276539,
                                      350664, 420399,
                                      464873, 507988,
                                      549466, 589487,
                                      627997, 601629,
                                      580965, 565776,
                                      555217),
                            "N10" = c(0, 113921,
                                      197854, 276862,
                                      351147, 421042,
                                      465716, 509005,
                                      550645, 590837,
                                      629533, 603328,
```

```
                                    582814, 567737,
                                    557220),
                          "N11" = c(873036, 758842,
                                    674423, 594702,
                                    519510, 448514,
                                    402627, 357978,
                                    314823, 273006,
                                    232543, 257023,
                                    275811, 289161,
                                    297930))

## variational inference for footprints in the data
inf <- infer_footprints_vb(cooc_ctable = ftp_5_10_data,
                           ftp_lengths = 2:15)

## get estimates and plot footprint spectrum
inference_summary_list <- get_ftp_inference_summary(inf, plot = TRUE)
```

---

`get_ftp_models_for_prediction` *Create parameters for* `predict_footprints` *and* `predict_footprints_SE` *from a footprint inference summary*

---

## Description

Utility function to generate a list of parameters required by `predict_footprints` or `predict_footprints_SE` using a footprint inference summary obtained from `get_ftp_inference_summary`. These parameters include footprint models, background protection probabilities, and coverage priors.

## Usage

```
get_ftp_models_for_prediction(infer_summary)
```

## Arguments

`infer_summary`
: A `list` returned by `get_ftp_inference_summary`, which must contain `"ESTIMATES"` and `"FTP_SUGGEST"`. If you wish to override the suggested footprints, you can replace the `"FTP_SUGGEST"` element with your own `n x 2` matrix.

## Value

A `list` containing the following elements:

`FTP_MODELS` Footprint models suitable for the `footprint_models` parameter in `predict_footprints`.

`bgprotectprob` Numeric value of background protection probability, corresponding to the `bgprotectprob` parameter in `predict_footprints`.

`bgcoverprior` Numeric value of background coverage prior, corresponding to the `bgcoverprior` parameter in `predict_footprints`.

`get_ftp_PWMs_for_prediction` *Create footprint models for* `predict_footprints` *from an inferred footprint spectrum*

## Description

Utility function to generate a list of footprint models required by `predict_footprints`, using a footprint spectrum inferred by `infer_footprints_vb` or `ftp_spectral_analysis_SE` and summarized with `get_ftp_inference_summary`. The models encode footprint lengths, emission probabilities, and coverage priors for downstream prediction.

## Usage

```
get_ftp_PWMs_for_prediction(
  ftp_spectrum,
  ftp_len_mat,
  bg_cover,
  ftp_protect_prob
)
```

## Arguments

`ftp_spectrum` A `data.frame` containing inferred abundances of footprints.

`ftp_len_mat` A `matrix` specifying footprint lengths:

**Column 1** Minimum footprint length.

**Column 2** Maximum footprint length.

**Column 3** Increment for generating lengths from minimum to maximum.

Each row corresponds to a footprint group, with row names interpreted as group labels. PWMs will be generated for all lengths in `seq(min, max, by)`.

`bg_cover` Numeric value indicating the estimated fraction of accessible positions in the SMF dataset (used as background coverage).

`ftp_protect_prob` Numeric value of the emission probability for protected positions within footprints.

## Value

A `list` of footprint models (position weight matrices) suitable for the `footprint_models` parameter in `predict_footprints` and `predict_footprints_SE`.

## get_SeqContext_control_beta_shapes_SE

*Fit Beta distribution shapes for positive and negative controls*

### Description

Fit Beta distribution shapes for positive and negative controls

### Usage

```
get_SeqContext_control_beta_shapes_SE(
  se,
  neg_control_sampleName,
  pos_control_sampleName,
  assayName = "mod_prob",
  min_n_data = 100
)
```

### Arguments

| | |
|---|---|
| se | A SummarizedExperiment object containing read-level data, such as that returned by SingleMoleculeGenomicsIO::readModBam, including modification probabilities. |
| neg_control_sampleName | Character scalar. Sample name for the negative control (i.e., an SMF experiment without MTase treatment). |
| pos_control_sampleName | Character scalar. Sample name for the positive control (i.e., an SMF experiment with MTase treatment on naked DNA). |
| assayName | Character scalar specifying the name of the assay in se that contains read-level modification probabilities. |
| min_n_data | Integer. Minimum number of data points required per sequence context. Contexts with fewer observations are merged into the "other" category. |

### Value

A list of two data.tables containing Beta distribution shape parameters for the positive and negative controls: positiveControlShapes and negativeControlShapes.

## infer_footprints_optim

*Find point estimates for footprint abundance using Stan optimization*

### Description

Uses rstan::optimizing to compute point estimates of footprint abundances by maximizing the joint posterior defined by the model.

WARNING: Although finding a posterior mode is fast, the returned point estimates are often far from the posterior means because the distribution is highly non-symmetric. Therefore, footprint spectral analysis using rstan::optimizing is unreliable unless a very strong and symmetric prior is used.

## Usage

```
infer_footprints_optim(
  cooc_ctable,
  ftp_lengths = 2:200,
  ftp_prior_cover = NULL,
  bg_prior_cover = 0.5,
  total_cnt_prior_dirich = NULL,
  ftp_bg_model = c("informative_prior", "bg_fixed", "ftp_bg_fixed"),
  bg_model_params = list(bg_protect_prob_fixed = 0.05, bg_protect_min = 0.01,
    bg_protect_max = 0.2, bg_protect_mean = 0.05, bg_protect_totcount = 100),
  ftp_model_params = list(ftp_protect_prob_fixed = 0.95, ftp_protect_min = 0.8,
    ftp_protect_max = 0.99, ftp_protect_mean = 0.95, ftp_protect_totcount = 100)
  ...
)
```

## Arguments

`cooc_ctable` A data frame containing columns `S`, `N00`, `N01`, `N10`, and `N11`, where `S` is the spacing and the four `N**` columns contain observed joint counts (00, 01, 10, 11) at that spacing. Such tables can be produced with `nomeR::count_joint_frequencies()`, `fetchNOMe::get_cooccurrence_ctable_from_bams()` or `footprintR::countSt`

`ftp_lengths` Numeric vector of footprint lengths for which abundances should be inferred. Length 1 is reserved for background and must not be included.

`ftp_prior_cover`
Numeric vector of expected (prior) coverages for the footprint lengths in `ftp_lengths`. These values define the prior mean of the Dirichlet distribution over footprint/background abundances. All elements must be in `[0, 1]`. Together with `bg_prior_cover`, they will be scaled to sum to 1 if necessary (with a warning).

`bg_prior_cover`
Numeric value in `[0, 1]` specifying the expected (prior) fraction of background (unprotected) positions. Used together with `ftp_prior_cover` to parameterize the prior Dirichlet distribution.

`total_cnt_prior_dirich`
Numeric value giving the total count parameter of the prior Dirichlet distribution. Higher values impose stronger prior concentration around the mean (i.e., stronger prior beliefs), while lower values yield a more diffuse prior.

`ftp_bg_model` Character string specifying the modeling strategy:

- `"informative_prior"` Infer `bg_protect_prob`, `ftp_protect_prob`, and footprint abundances.
- `"bg_fixed"` Fix `bg_protect_prob` to the value in `bg_model_params[["bg_protect_` and infer `ftp_protect_prob` and footprint abundances.
- `"ftp_bg_fixed"` Fix both `bg_protect_prob` and `ftp_protect_prob` (values in `bg_model_params` and `ftp_model_params`). Only footprint abundances are inferred.

`bg_model_params`
A list of background model parameters:

- **bg_protect_prob_fixed** Fixed value for `bg_protect_prob` used in `"bg_fixed"` and `"ftp_bg_fixed"` models.
- **bg_protect_min, bg_protect_max** Lower and upper bounds for `bg_protect_prob` when it is inferred. Ignored in fixed models.

**bg_protect_mean** Mean of the prior Beta distribution for `bg_protect_prob`. Ignored when fixed.

**bg_protect_totcount** Total count parameter of the prior Beta distribution (controls concentration). Ignored when fixed.

`ftp_model_params`

A list of footprint model parameters:

**ftp_protect_prob_fixed** Fixed value for `ftp_protect_prob` (used only in `"ftp_bg_fixed"`).

**ftp_protect_min, ftp_protect_max** Bounds for `ftp_protect_prob` when it is inferred. Ignored in `"ftp_bg_fixed"`.

**ftp_protect_mean** Mean of the prior Beta distribution for `ftp_protect_prob`. Ignored when fixed.

**ftp_protect_totcount** Total count parameter of the prior Beta distribution. Ignored when fixed.

`...` Additional arguments passed to `optimizing`. See the `rstan` documentation for available parameters.

## Value

A list returned by `optimizing`, containing the optimization results. The attribute `attr(<list>, "ftp_lengths")` stores the vector of footprint lengths used for inference (i.e., the user-supplied `ftp_lengths` parameter).

## Examples

```
## Simple data with two footprints of lengths 5 and 10 bps.
## The table below is a count table of observed occurrences of
## 00, 01, 10, and 11 at spacings from 1 until 15.

ftp_5_10_data <- data.frame("S" = 1:15,
                            "N00" = c(1626964, 1508381,
                                      1420066, 1336897,
                                      1258679, 1185045,
                                      1136784, 1090029,
                                      1045066, 1001670,
                                      959927, 983020,
                                      1000410, 1012326,
                                      1019633),
                            "N01" = c(0, 113856,
                                      197657, 276539,
                                      350664, 420399,
                                      464873, 507988,
                                      549466, 589487,
                                      627997, 601629,
                                      580965, 565776,
                                      555217),
                            "N10" = c(0, 113921,
                                      197854, 276862,
                                      351147, 421042,
                                      465716, 509005,
                                      550645, 590837,
                                      629533, 603328,
                                      582814, 567737,
                                      557220),
```

```
                                "N11" = c(873036, 758842,
                                          674423, 594702,
                                          519510, 448514,
                                          402627, 357978,
                                          314823, 273006,
                                          232543, 257023,
                                          275811, 289161,
                                          297930))


## finding MAP estimate
inf_output <- infer_footprints_optim(cooc_ctable = ftp_5_10_data,
                                     ftp_lengths = 2:15)

## plot footprint spectrum
get_ftp_inference_summary(inf_output, plot = TRUE)
```

---

`infer_footprints_sampling`
*Footprint spectral analysis using the No-U-Turn Sampler (NUTS) implemented in Stan*

---

## Description

Performs Bayesian inference of footprint abundances in single-molecule footprinting data (e.g., NOMe-seq) using `rstan`'s implementation of Hamiltonian Monte Carlo (HMC) with the No-U-Turn Sampler (NUTS). The algorithm draws samples from the posterior distribution defined by the underlying Bayesian model.

WARNING: Although HMC sampling is unbiased, it is very slow for the range of footprint lengths typical in real-world SMF datasets.

## Usage

```
infer_footprints_sampling(
  cooc_ctable,
  ftp_lengths = 2:200,
  ftp_prior_cover = NULL,
  bg_prior_cover = 0.5,
  total_cnt_prior_dirich = NULL,
  ftp_bg_model = c("informative_prior", "bg_fixed", "ftp_bg_fixed"),
  bg_model_params = list(bg_protect_prob_fixed = 0.05, bg_protect_min = 0.01,
    bg_protect_max = 0.2, bg_protect_mean = 0.05, bg_protect_totcount = 100),
  ftp_model_params = list(ftp_protect_prob_fixed = 0.95, ftp_protect_min = 0.8,
    ftp_protect_max = 0.99, ftp_protect_mean = 0.95, ftp_protect_totcount = 100),
  nchains = 4,
  ncpu = nchains,
  rstan_control = list(max_treedepth = 15, adapt_delta = 0.9),
  ...
)
```

## Arguments

`cooc_ctable` A data frame containing columns `S`, `N00`, `N01`, `N10`, and `N11`, where `S` is the spacing and the four `N**` columns contain observed joint counts (00, 01, 10, 11) at that spacing. Such tables can be produced with `nomeR::count_joint_frequencies()`, `fetchNOMe::get_cooccurrence_ctable_from_bams()` or `footprintR::countSt`

`ftp_lengths` Numeric vector of footprint lengths for which abundances should be inferred. Length 1 is reserved for background and must not be included.

`ftp_prior_cover` Numeric vector of expected (prior) coverages for the footprint lengths in `ftp_lengths`. These values define the prior mean of the Dirichlet distribution over footprint/background abundances. All elements must be in `[0, 1]`. Together with `bg_prior_cover`, they will be scaled to sum to 1 if necessary (with a warning).

`bg_prior_cover` Numeric value in `[0, 1]` specifying the expected (prior) fraction of background (unprotected) positions. Used together with `ftp_prior_cover` to parameterize the prior Dirichlet distribution.

`total_cnt_prior_dirich` Numeric value giving the total count parameter of the prior Dirichlet distribution. Higher values impose stronger prior concentration around the mean (i.e., stronger prior beliefs), while lower values yield a more diffuse prior.

`ftp_bg_model` Character string specifying the modeling strategy:

- `"informative_prior"` Infer `bg_protect_prob`, `ftp_protect_prob`, and footprint abundances.
- `"bg_fixed"` Fix `bg_protect_prob` to the value in `bg_model_params[["bg_protect_` and infer `ftp_protect_prob` and footprint abundances.
- `"ftp_bg_fixed"` Fix both `bg_protect_prob` and `ftp_protect_prob` (values in `bg_model_params` and `ftp_model_params`). Only footprint abundances are inferred.

`bg_model_params` A list of background model parameters:

- **bg_protect_prob_fixed** Fixed value for `bg_protect_prob` used in `"bg_fixed"` and `"ftp_bg_fixed"` models.
- **bg_protect_min, bg_protect_max** Lower and upper bounds for `bg_protect_prob` when it is inferred. Ignored in fixed models.
- **bg_protect_mean** Mean of the prior Beta distribution for `bg_protect_prob`. Ignored when fixed.
- **bg_protect_totcount** Total count parameter of the prior Beta distribution (controls concentration). Ignored when fixed.

`ftp_model_params` A list of footprint model parameters:

- **ftp_protect_prob_fixed** Fixed value for `ftp_protect_prob` (used only in `"ftp_bg_fixed"`).
- **ftp_protect_min, ftp_protect_max** Bounds for `ftp_protect_prob` when it is inferred. Ignored in `"ftp_bg_fixed"`.
- **ftp_protect_mean** Mean of the prior Beta distribution for `ftp_protect_prob`. Ignored when fixed.
- **ftp_protect_totcount** Total count parameter of the prior Beta distribution. Ignored when fixed.

| | |
|---|---|
| nchains | Integer specifying the number of Markov chains to run. |
| ncpu | Integer specifying the number of CPU cores to use. |
| rstan_control | A list of control parameters passed to the Stan sampler. |
| ... | Additional arguments passed to sampling. See the rstan documentation for details. |

## Value

A stanfit object (S4 class) containing the posterior samples. See sampling for details. The attribute attr(<stanfit_object>, "ftp_lengths") contains the vector of footprint lengths used for inference (i.e., the user-supplied ftp_lengths parameter).

## Examples

```
## Simple data with two footprints of lengths 5 and 10 bps.
## The table below is a count table of observed occurrences of
## 00, 01, 10, and 11 at spacings from 1 until 15.

ftp_5_10_data <- data.frame("S" = 1:15,
                            "N00" = c(1626964, 1508381,
                                      1420066, 1336897,
                                      1258679, 1185045,
                                      1136784, 1090029,
                                      1045066, 1001670,
                                      959927, 983020,
                                      1000410, 1012326,
                                      1019633),
                            "N01" = c(0, 113856,
                                      197657, 276539,
                                      350664, 420399,
                                      464873, 507988,
                                      549466, 589487,
                                      627997, 601629,
                                      580965, 565776,
                                      555217),
                            "N10" = c(0, 113921,
                                      197854, 276862,
                                      351147, 421042,
                                      465716, 509005,
                                      550645, 590837,
                                      629533, 603328,
                                      582814, 567737,
                                      557220),
                            "N11" = c(873036, 758842,
                                      674423, 594702,
                                      519510, 448514,
                                      402627, 357978,
                                      314823, 273006,
                                      232543, 257023,
                                      275811, 289161,
                                      297930))


## HMC inference for footprints in the data
inf_output <- infer_footprints_sampling(cooc_ctable = ftp_5_10_data,
```

```
                                        ftp_lengths = 2:15, ncpu = 2)

## plot footprint spectrum
get_ftp_inference_summary(inf_output, plot = TRUE)
```

---

`infer_footprints_vb` *Footprint spectral analysis by Variational Bayes inference using Stan Automatic Differentiation Variational Inference (ADVI)*

---

## Description

Performs Bayesian inference of footprint abundances using Stan's Automatic Differentiation Variational Inference (ADVI) algorithm. The method fits a probabilistic model to joint co-occurrence counts across genomic spacings, allowing estimation of background and footprint-specific protection probabilities and footprint abundances.

## Usage

```
infer_footprints_vb(
  cooc_ctable,
  ftp_lengths = 2:200,
  ftp_prior_cover = NULL,
  bg_prior_cover = 0.5,
  total_cnt_prior_dirich = NULL,
  ftp_bg_model = c("informative_prior", "bg_fixed", "ftp_bg_fixed"),
  bg_model_params = list(bg_protect_prob_fixed = 0.05, bg_protect_min = 0.01,
    bg_protect_max = 0.2, bg_protect_mean = 0.05, bg_protect_totcount = 100),
  ftp_model_params = list(ftp_protect_prob_fixed = 0.95, ftp_protect_min = 0.8,
    ftp_protect_max = 0.99, ftp_protect_mean = 0.95, ftp_protect_totcount = 100)
  max_nruns = 3,
  max_pareto_k = 10,
  iter = 15000,
  tol_rel_obj = 1e-08,
  output_samples = 2000,
  grad_samples = 1,
  algorithm = "meanfield",
  ...
)
```

## Arguments

| | |
|---|---|
| `cooc_ctable` | A data frame containing columns `S`, `N00`, `N01`, `N10`, and `N11`, where `S` is the spacing and the four `N**` columns contain observed joint counts (00, 01, 10, 11) at that spacing. Such tables can be produced with `nomeR::count_joint_frequencies()`, `fetchNOMe::get_cooccurrence_ctable_from_bams()` or `footprintR::countSt` |
| `ftp_lengths` | Numeric vector of footprint lengths for which abundances should be inferred. Length 1 is reserved for background and must not be included. |

ftp_prior_cover
: Numeric vector of expected (prior) coverages for the footprint lengths in ftp_lengths. These values define the prior mean of the Dirichlet distribution over footprint/background abundances. All elements must be in [0, 1]. Together with bg_prior_cover, they will be scaled to sum to 1 if necessary (with a warning).

bg_prior_cover
: Numeric value in [0, 1] specifying the expected (prior) fraction of background (unprotected) positions. Used together with ftp_prior_cover to parameterize the prior Dirichlet distribution.

total_cnt_prior_dirich
: Numeric value giving the total count parameter of the prior Dirichlet distribution. Higher values impose stronger prior concentration around the mean (i.e., stronger prior beliefs), while lower values yield a more diffuse prior.

ftp_bg_model
: Character string specifying the modeling strategy:

  "informative_prior" Infer bg_protect_prob, ftp_protect_prob, and footprint abundances.

  "bg_fixed" Fix bg_protect_prob to the value in bg_model_params[["bg_protect_ and infer ftp_protect_prob and footprint abundances.

  "ftp_bg_fixed" Fix both bg_protect_prob and ftp_protect_prob (values in bg_model_params and ftp_model_params). Only footprint abundances are inferred.

bg_model_params
: A list of background model parameters:

  **bg_protect_prob_fixed** Fixed value for bg_protect_prob used in "bg_fixed" and "ftp_bg_fixed" models.

  **bg_protect_min, bg_protect_max** Lower and upper bounds for bg_protect_prob when it is inferred. Ignored in fixed models.

  **bg_protect_mean** Mean of the prior Beta distribution for bg_protect_prob. Ignored when fixed.

  **bg_protect_totcount** Total count parameter of the prior Beta distribution (controls concentration). Ignored when fixed.

ftp_model_params
: A list of footprint model parameters:

  **ftp_protect_prob_fixed** Fixed value for ftp_protect_prob (used only in "ftp_bg_fixed").

  **ftp_protect_min, ftp_protect_max** Bounds for ftp_protect_prob when it is inferred. Ignored in "ftp_bg_fixed".

  **ftp_protect_mean** Mean of the prior Beta distribution for ftp_protect_prob. Ignored when fixed.

  **ftp_protect_totcount** Total count parameter of the prior Beta distribution. Ignored when fixed.

max_nruns
: Maximum number of attempts to run vb. Some initializations may fail to converge, in which case multiple attempts are made.

max_pareto_k
: Maximum allowable pareto_k value returned by vb. If exceeded, inference is retried (up to max_nruns times).

iter, tol_rel_obj, output_samples, grad_samples, algorithm, ...
: Additional arguments passed to vb, controlling ADVI inference. See vb for details.

## Value

A `stanfit` object containing the ADVI posterior approximation. The attribute `attr(<stanfit_object>, "ftp_lengths")` stores the footprint lengths used for inference (i.e., the supplied `ftp_lengths`).

## Examples

```
## Simple data with two footprints of lengths 5 and 10 bps.
## The table below is a count table of observed occurrences of
## 00, 01, 10, and 11 at spacings from 1 until 15.

ftp_5_10_data <- data.frame("S" = 1:15,
                            "N00" = c(1626964, 1508381,
                                      1420066, 1336897,
                                      1258679, 1185045,
                                      1136784, 1090029,
                                      1045066, 1001670,
                                      959927, 983020,
                                      1000410, 1012326,
                                      1019633),
                            "N01" = c(0, 113856,
                                      197657, 276539,
                                      350664, 420399,
                                      464873, 507988,
                                      549466, 589487,
                                      627997, 601629,
                                      580965, 565776,
                                      555217),
                            "N10" = c(0, 113921,
                                      197854, 276862,
                                      351147, 421042,
                                      465716, 509005,
                                      550645, 590837,
                                      629533, 603328,
                                      582814, 567737,
                                      557220),
                            "N11" = c(873036, 758842,
                                      674423, 594702,
                                      519510, 448514,
                                      402627, 357978,
                                      314823, 273006,
                                      232543, 257023,
                                      275811, 289161,
                                      297930))


## VB inference for footprints in the data
inf_output <- infer_footprints_vb(cooc_ctable = ftp_5_10_data,
                                  ftp_lengths = 2:15)

## plot footprint spectrum
get_ftp_inference_summary(inf_output, plot = TRUE)
```

## plot_ftp_spectra_DF

*Plot footprint spectra from a DataFrame*

### Description

Generates a panel of footprint spectrum plots for all samples in a `DataFrame` obtained from `ftp_spectral_analysis_SE`. The input `DataFrame` must contain a column named `ftp_spectrum`.

### Usage

```
plot_ftp_spectra_DF(DF)
```

### Arguments

| | |
|---|---|
| `DF` | A `DataFrame` containing footprint spectra for each sample. Must include a column `ftp_spectrum` as returned by `ftp_spectral_analysis_SE`. |

### Value

A `patchwork` object containing a panel of footprint spectrum plots, one for each sample in `DF`.

## plot_ftp_spectrum *Plot a footprint abundance spectrum*

### Description

Generates a `ggplot2` plot of inferred footprint abundances across different footprint lengths. The plot visualizes both the mean coverage estimates and a measure of statistical significance of estimated values.

### Usage

```
plot_ftp_spectrum(ftp_spectrum, title = NULL)
```

### Arguments

| | |
|---|---|
| `ftp_spectrum` | A `data.frame` containing inferred footprint abundances. Must include the columns `ftp_length` and `mean`. |
| `title` | Optional character string to use as the plot title. |

### Value

A `ggplot2` object.

- The **dark green line** (left Y-axis, log10 scale) represents the estimated coverage (mean across samples from the posterior distribution) for each footprint length on the X-axis.
- The **red line** (right Y-axis, linear scale) represents the ratio of mean to standard deviation across samples for each footprint length, which can be interpreted as approximate Z-scores. These values can be used to assess the statistical significance of the inferred footprint coverage estimates.

`predict_footprints` *Calculate posterior probabilities and predict footprints in single-molecule footprinting (SMF) data*

## Description

Computes posterior probabilities and predicts footprint configurations in single-molecule footprinting (SMF) datasets.

## Usage

```
predict_footprints(
  data,
  footprint_models,
  bgprotectprob,
  bgcoverprior,
  aggrByGroup = TRUE,
  ftpConfigMethod = c("PV", "PosteriorDecoding", "Viterbi"),
  keepStartProb = FALSE,
  ncpu = 1L,
  verbose = FALSE
)
```

## Arguments

`data` A `matrix` or `list` containing binarized SMF data.

- If a `matrix`, rows correspond to reads (or fragments) and columns correspond to positions in the region of interest (ROI).
- If a `list`, each element must be a vector containing SMF data. Data must be binarized: '0' represents accessible (methylated) positions, '1' represents protected (unmethylated) positions, and all other positions should be `NA`.

`footprint_models`

A list of footprint models for proteins. Each element must contain:

**PROTECT_PROB** Numeric vector of footprint emission probabilities for protected positions within a footprint.

**COVER_PRIOR** Numeric prior probability reflecting the expected fraction of reads covered by the footprint.

**NAME** Unique name of the model (character), e.g., "Nucleosome–149", "Nucleosome–150".

**GROUP** Optional non-unique group name (character) used for aggregating probabilities if `aggrByGroup = TRUE`. For example, footprints with names "Nucleosome–149" and "Nucleosome–150" may share the GROUP "Nucleosome".

NOTE: To account for very short footprints that likely reflect correlated noise within accessible regions, users can define short footprints (usually 2–5 bp) and set `GROUP` to "background". In this case, if `aggrByGroup` is `TRUE`, the posteriors for background will additionally be aggregated across these short footprints.

bgprotectprob
: Background emission probability of a protected position in open (accessible) regions.

bgcoverprior
: Prior probability of a position being in the free (accessible/background) state.

aggrByGroup
: Logical. If TRUE, probabilities are aggregated by the GROUP ID in footprint_models. If FALSE, or if GROUP IDs are missing, probabilities are reported for each individual footprint NAME.

ftpConfigMethod
: Algorithm for decoding footprint configurations:

  PV Posterior-Viterbi (default): uses footprint coverage posteriors and a Viterbi-like algorithm to find a valid footprint configuration maximizing posterior coverage probability (see Fariselli et al, 2005). The score for each footprint is the geometric mean of posterior coverages across all positions covered by the footprint.

  PosteriorDecoding Posterior decoding using calculated coverage posteriors. The algorithm returns segments where coverage posteriors for a given footprint are maximum compared to other footprints. If coverage posteriors are aggregated by GROUP, i.e. aggrByGroup is TRUE, the ftp_name and ftp_group are identical and correspond to groups defined in footprint_models. NOTE: The widths of the segments do not necessarily match the footprint lengths defined by footprint_models. The score for each segment is the geometric mean of posterior coverage for the corresponding footprint across all positions within the segment.

  Viterbi Classic Viterbi algorithm to find the most probable footprint configuration. The score corresponds to the posterior start probability for each reported footprint.

keepStartProb
: Logical. If TRUE, posteriors for footprint start positions are included in the return value.

ncpu
: Number of threads to use.

verbose
: Logical. If TRUE, enables verbose output for debugging.

## Value

A list containing three data frames:

COVER_PROB Contains coverage probabilities for each SMF molecule (seq), each position in ROI (pos), and each footprint model. These probabilities indicate how likely a position is covered by a given footprint.

FOOTPRINT_CONF Contains footprint configurations predicted for each molecule using the selected ftpConfigMethod. Each row reports the SMF molecule (seq), start position (start), width (width), footprint name (ftp_name), group (ftp_group), and confidence score (score) between 0 and 1.

START_PROB **(optional, if** keepStartProb = TRUE**.)** Contains start probabilities for each SMF molecule (seq), each position in ROI (pos), and each footprint model. These probabilities indicate how likely a footprint starts at each position in a fragment.

## References

Fariselli, P., Martelli, P. L., & Casadio, R. (2005). *A new decoding algorithm for Hidden Markov Models improves the prediction of the topology of all-beta membrane proteins.* BMC Bioinformatics, 6(S4), S12. https://doi.org/10.1186/1471-2105-6-S4-S12

## Examples

```
set.seed(3346)
nc <- 50
nr <- 50
rmatr <- matrix(data = as.integer(rnorm(nc * nr) >= 0.5),
                ncol = nc,nrow=nr)

## create dummy footprints
bg.pr <- 0.5
ft.pr <- 1-bg.pr
ft.len <- 15

## creating a list of binding models for nomeR
ftp.models <- list(list("PROTECT_PROB" = rep(0.99,ft.len),
                        "COVER_PRIOR" = ft.pr,
                        "NAME" = "FOOTPRINT"))

nomeR.out <- predict_footprints(data=rmatr,
                                footprint_models = ftp.models,
                                bgprotectprob = 0.05,
                                bgcoverprior = bg.pr)
```

---

`predict_footprints_SE`

*Calculate posterior probabilities and predict footprints in a SummarizedExperiment containing single-molecule footprinting (SMF) data*

---

## Description

Computes posterior probabilities and predicts footprint configurations for SMF data stored in a `SummarizedExperiment` object.

## Usage

```
predict_footprints_SE(
  se,
  assayName = "mod_prob",
  threshMod = 0.5,
  threshUnmod = threshMod,
  min_frag_data_len = 50L,
  min_frag_data_dens = 0.05,
  footprint_models,
  bgprotectprob,
  bgcoverprior,
  aggrByGroup = TRUE,
  keepStartProb = FALSE,
  ftpConfigMethod = c("PV", "PosteriorDecoding", "Viterbi"),
  returnAs = c("SE", "data.table"),
  ncpu = 1L,
  verbose = FALSE,
  profile = FALSE
)
```

## Arguments

| | |
|---|---|
| se | A SummarizedExperiment object containing read-level data, such as that returned by SingleMoleculeGenomicsIO::readModBam, including modification probabilities. |
| assayName | Character scalar specifying the name of the assay in se that contains read-level modification probabilities. |
| threshUnmod, threshMod | Numeric thresholds used to binarize modification probabilities into accessible (0; probability >= threshMod), protected (1; probability < threshUnmod), or unknown (NA) states. |
| min_frag_data_len | Ignore fragments whose genomic span (from the leftmost to rightmost non-NA data point) is shorter than min_frag_data_len. |
| min_frag_data_dens | Ignore fragments with a density of informative (non-NA) positions below min_frag_data_dens. |
| footprint_models | A list of footprint models for proteins. Each element must contain:<br>**PROTECT_PROB** Numeric vector of footprint emission probabilities for protected positions within a footprint.<br>**COVER_PRIOR** Numeric prior probability reflecting the expected fraction of reads covered by the footprint.<br>**NAME** Unique name of the model (character), e.g., "Nucleosome–149", "Nucleosome–150".<br>**GROUP** Optional non-unique group name (character) used for aggregating probabilities if aggrByGroup = TRUE. For example, footprints with names "Nucleosome–149" and "Nucleosome–150" may share the GROUP "Nucleosome".<br>NOTE: To account for very short footprints that likely reflect correlated noise within accessible regions, users can define short footprints (usually 2–5 bp) and set GROUP to "background". In this case, if aggrByGroup is TRUE, the posteriors for background will additionally be aggregated across these short footprints. |
| bgprotectprob | Background emission probability of a protected position in open (accessible) regions. |
| bgcoverprior | Prior probability of a position being in the free (accessible/background) state. |
| aggrByGroup | Logical. If TRUE, probabilities are aggregated by the GROUP ID in footprint_models. If FALSE, or if GROUP IDs are missing, probabilities are reported for each individual footprint NAME. |
| keepStartProb | Logical. If TRUE, posteriors for footprint start positions are included in the return value. |
| ftpConfigMethod | Algorithm for decoding footprint configurations:<br>PV Posterior-Viterbi (default): uses footprint coverage posteriors and a Viterbi-like algorithm to find a valid footprint configuration maximizing posterior coverage probability (see Fariselli et al, 2005). The score for each footprint is the geometric mean of posterior coverages across all positions covered by the footprint. |

PosteriorDecoding Posterior decoding using calculated coverage posteriors. The algorithm returns segments where coverage posteriors for a given footprint are maximum compared to other footprints. If coverage posteriors are aggregated by GROUP, i.e. aggrByGroup is TRUE, the ftp_name and ftp_group are identical and correspond to groups defined in footprint_models. NOTE: The widths of the segments do not necessarily match the footprint lengths defined by footprint_models. The score for each segment is the geometric mean of posterior coverage for the corresponding footprint across all positions within the segment.

Viterbi Classic Viterbi algorithm to find the most probable footprint configuration. The score corresponds to the posterior start probability for each reported footprint.

returnAs Character scalar. Return output as a SummarizedExperiment ("SE") or as a list of data.tables ("data.table").

ncpu Number of threads to use.

verbose Logical. If TRUE, enables verbose output for debugging.

profile Logical. If TRUE, enables time profiling of internal steps.

## Value

If returnAs="SE" - a SummarizedExperiment object containing:

- the original mod_prob assay,
- additional assays with calculated posterior coverage probabilities (and start probabilities if keepStartProb = TRUE) (e.g. "Nucl_coverProb_nomeR"), and
- predicted footprint configurations stored as IntegerList objects in colData (e.g. column "Nucl_nomeR").

If returnAs="data.table" - a list containing data.tables for:

COVER_PROB Contains coverage probabilities for each SMF molecule (fragID) and each footprint model. These probabilities indicate how likely a position is covered by a given footprint. The column mod_prob reports the original modification probabilities from the input se object.

FOOTPRINT_CONF Contains footprint configurations (decoding) predicted for each molecule using the selected ftpConfigMethod. Each row reports the SMF molecule (fragID), start position (start), width (width), footprint name (ftp_name), group (ftp_group), and confidence score (score) between 0 and 1.

START_PROB **(optional, if** keepStartProb = TRUE**.)** Contains start probabilities for each SMF molecule (fragID) and each footprint model. These probabilities indicate how likely a footprint starts at each position in a fragment.

seqnames, start and strand are genomic coordinates in the reference.

## References

Fariselli, P., Martelli, P. L., & Casadio, R. (2005). *A new decoding algorithm for Hidden Markov Models improves the prediction of the topology of all-beta membrane proteins.* BMC Bioinformatics, 6(S4), S12. https://doi.org/10.1186/1471-2105-6-S4-S12

`suggest_footprints` *Suggest potential footprints from an inferred spectrum*

## Description

Identifies candidate footprints based on an inferred footprint abundance spectrum. The function smooths the spectrum using `smooth.spline` to detect local extrema. Local maxima are taken as potential footprints, and peaks are extended until the signal decreases by `max_abund_log2drop`, reaches a local minimum, or exceeds `max_peak_width`.

## Usage

```
suggest_footprints(
  S,
  y,
  max_abund_log2drop = 1,
  max_peak_width = Inf,
  isLog = TRUE,
  spline_spar = 0.6,
  ...
)
```

## Arguments

| | |
|---|---|
| `S` | Numeric vector of footprint lengths, as returned by `infer_footprints_vb`. Note that `S = 1` is reserved for background and should generally be removed before running this function. |
| `y` | Numeric vector of mean footprint abundances (or another measure) corresponding to `S`, as returned by `infer_footprints_vb`. |
| `max_abund_log2drop` | Numeric specifying the maximum decrease in abundance (relative to the local maximum) until peaks are no longer extended. |
| `max_peak_width` | Numeric specifying the maximum width of peaks in the spectrum. |
| `isLog` | Logical indicating whether `y` is log-scaled. |
| `spline_spar` | Numeric in (0, 1], controlling the smoothness of the spline (`smooth.spline`). Higher values produce smoother curves. Recommended values to test include 0.1, 0.3, 0.5, 0.75. |
| `...` | Additional parameters passed to `smooth.spline`. |

## Value

A `list` containing:

`ftp_ranges` A `matrix` with suggested footprint ranges.

`smoothed_signal` The smoothed spectrum, as returned by `smooth.spline`, evaluated at each value of `S`.

`write_ftp_decoding_bed` *Write footprint decoding to a BED12 file*

## Description

Note that the molecule coordinates in the output BED12 file span the region from the start of the leftmost to the end of the rightmost footprint and do not correspond to the original genomic coordinates of the sequenced molecules.

## Usage

```
write_ftp_decoding_bed(ftp_decode_dt, file, ...)
```

## Arguments

| | |
|---|---|
| `ftp_decode_dt` | A `data.table` with footprint decoding results, as returned by `predict_footprints_SE` with `returnAs="data.table"`. |
| `file` | Character scalar. Path to the output BED12 file. |
| `...` | Additional options passed to `fwrite`. |

# Index